Prezime i ime učenika : *Stoiljković Mihajlo*

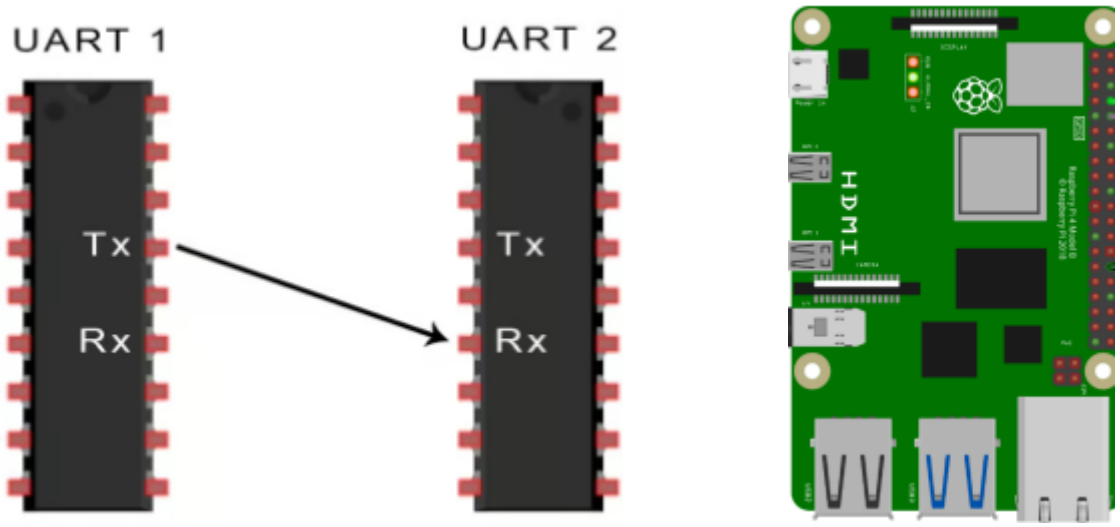
Odeljenje i grupa : *IV-4 treca grupa*

R. broj i naziv vežbe : *Vežba 7. Komunikacija mikrokontrolera sa drugim računarom*

IZVEŠTAJ:

Ova vežba se realizuje korišćenjem jednog mikrokontrolera **Arduino UNO** i mikroračunara **Raspberry Pi 4 (400)**. Cilj jeste da oba uređaja šalju i primaju podatke međusobno tako što se sa mikroračunara šalje signal kao korisnički unos na Arduino mikrokontroler koji na osnovu unosa vrši uključivanje/isključivanje LED diode. Pored te mogućnosti, imamo takođe i slanje statusa gde se svi ti parametri odnosno stanja ispisuju u serijski monitor kod **Arduino IDE** razvojnog okruženja.

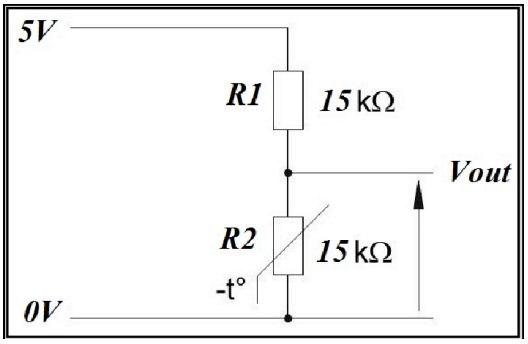
Za komunikaciju između ova dva uređaja se koristi UART serijska komunikacija koja kao što je poznato koristi TX i RX pinove. Kod arduina su to digitalni pinovi 0 i 1 koji su rezervisani za UART serijsku komunikaciju, dok kod mikroracunara se koriste **GPIO 14** za TX i **GPIO 15** za RX pin.

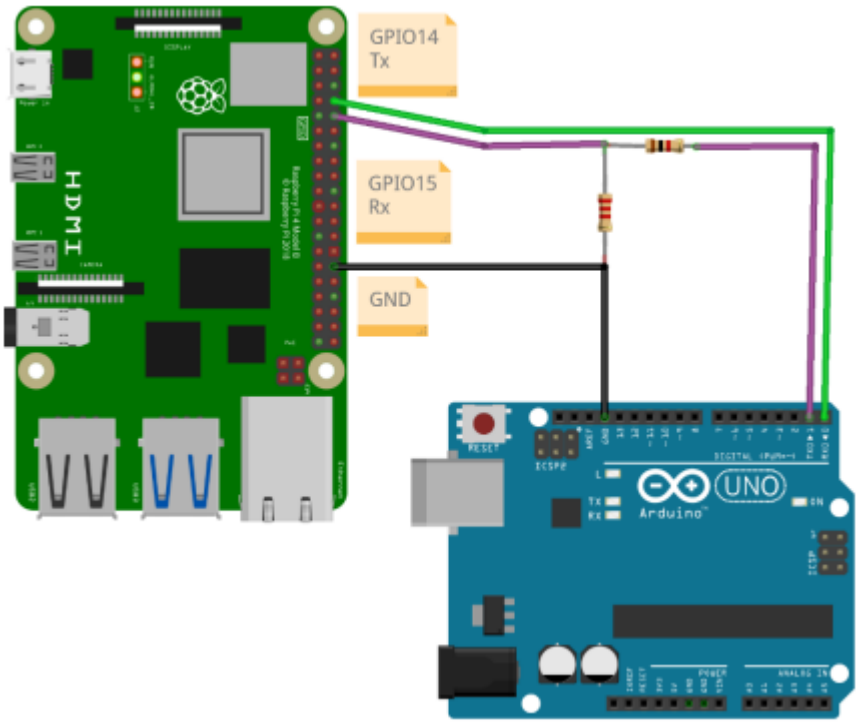


Gore na slici je prikazan izgled Raspberry Pi mikroračunara. Model koji se koristi u vežbi nije isti, s obzirom da je PCB drugačiji i ima kao kuciste tastaturu, mada je funkcionalnost ista.



Važno je napomenuti da Raspberry Pi koristi 3.3V napon dok Arduino najčešće koristi 5V, pa s tim nije preporučljivo direktno povezivati TX i RX linije, već koristiti razdelnih napona za Arduino TX i RPi RX. Potrebno je koristiti otpornik od 1kΩ kao i otpornik od 2kΩ. Ovime smo dobili da sa Arduina ne ide direktno 5V na Raspberry pi, već uz pomoć razdelnika napona taj napon bude približno 3V i može se bezbedno koristiti kod mikroračunara. Šema povezivanja je prikazana na slici.





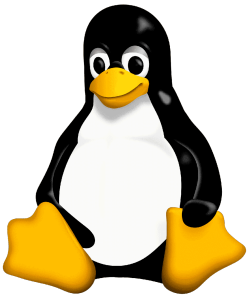
Nakon povezivanja, potrebno je konfigurisati UART interfejs kod mikroračunara. S obrizom da mikroračunar koristi Raspberry Pi OS koji je baziran na Debian linux operativnom sistemu, uz pomoć terminala unosimo komande koje su bazirane na **sudo (super user do)** koji predstavlja super korisnicki pristup što znači da se imaju najveće privilegije nad operativnim sistemom. U terminalu kucamo komande

sudo raspi-config

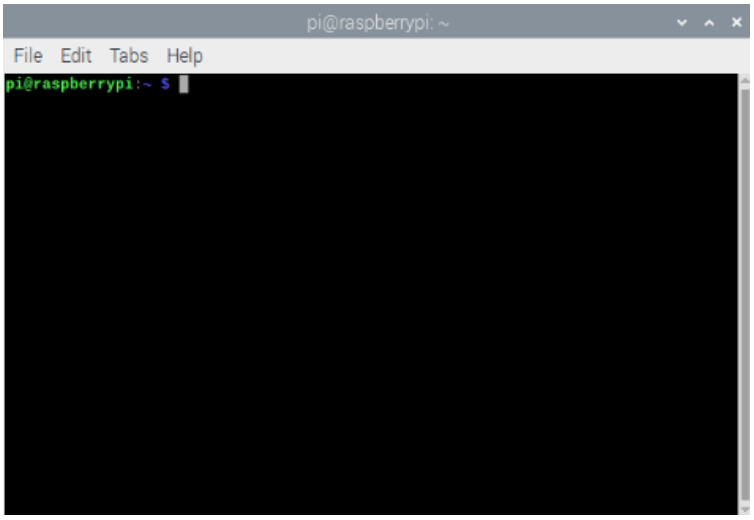
Interface Options → Serial Port

“Login shell over serial?” → NO

“Enable serial hardware?” → YES



Nakon toga, poželjno je izvršiti resetovanje mikroračunara, što možemo izvršiti tako što koristimo grafički korisnički interfejs operativnog sistema, ili koristiti terminal i napisati komandu reboot. Ovo podešavanje smo izvršili zato što linux nativno ne podržava korišćenje serijske komunikacije bez podešavanja iste. Izgled linux terminala prikazan je na slici ispod.



Nakon što smo izvršili konfigurisanje Raspberry PI operativnog sistema i povezali električnu šemu, sledi programiranje mikrokontrolera i mikroračunara. Arduino kod:

```
void setup() {
  Serial.begin(9600);
  pinMode(13, OUTPUT);
}
void loop() {
  // Slanje periodičnog statusa
  Serial.println("ARDUINO_OK");
  delay(1000);
  // Čitanje komandi
  if (Serial.available()) {
    String cmd = Serial.readStringUntil('\n');
    cmd.trim();
    if (cmd == "LED ON") {
      digitalWrite(13, HIGH);
      Serial.println("ACK: LED ON");
    }
    else if (cmd == "LED OFF") {
      digitalWrite(13, LOW);
      Serial.println("ACK: LED OFF");
    }
    else if (cmd == "STATUS?") {
      Serial.println("STATUS: LED=" + String(digitalRead(13)));
    }
    else {
      Serial.println("ERR: Unknown CMD");
    }
  }
}
```

Opis Arduino (c/c++) programa



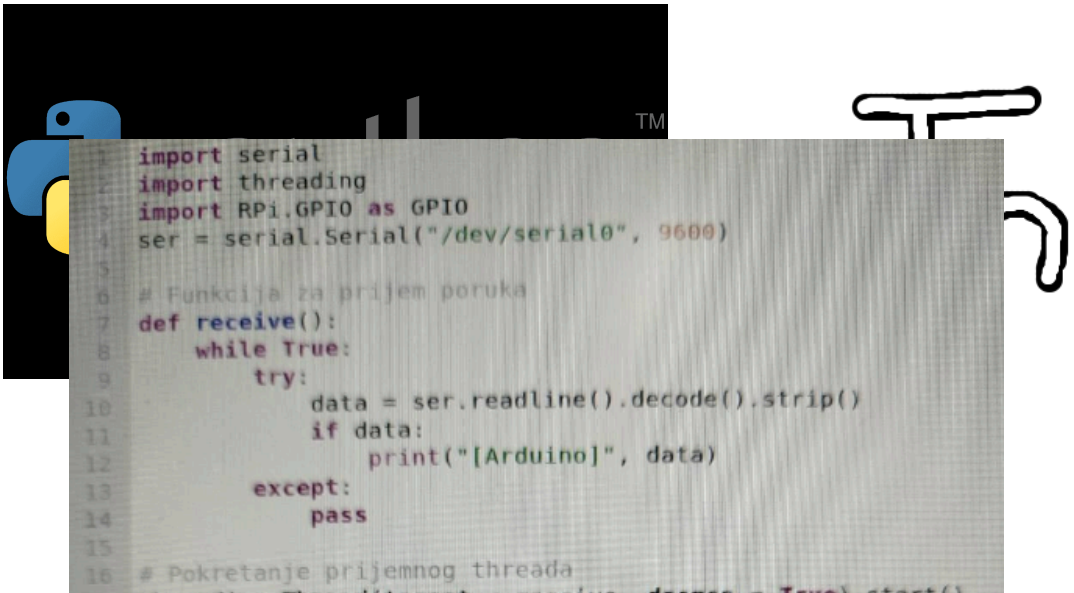
Cilj Arduino programa je da čeka komandu od mikroračunara (LED ON, LED OFF, STATUS?) kao string, odgovori tekstualnim porukama i šalje periodične statuse. Ovde se koristi ugrađena dioda na samoj ploči Arduina. Naznačena je oznakom L na ploči i koristi digitalni pin 13. Ostale dve diode blinkaju na osnovu toga kada se šalje TX signal i prima RX signal.

U setup() funkciji se vrši inicijalizacija serijskog monitora kao i postavljanje pina 13 kao izlazni odnosno OUTPUT.

U loop() funkciji se na početku vrši ispis u serijskom monitoru “ARDUINO_OK” na svaku jednu sekundu kako bi se prikazivalo kod mikroračunara da je komunikacija uspešna. Nakon toga sledi uslov koji proverava da li je serijski monitor dostupan, gde ukoliko jeste vrši se inicijalizacija i deklaracija stringa cmd koji čita iz serijskog monitora dok tekst ne predje u novu liniju. Nakon toga taj string koristi trim funkciju koja uklanja whitespace odnosno razmake i prazna mesta u stringu. Posle ovog stringa slede ugnjezdeni uslovi odnosno uslovi u uslovu gde se proverava vrednost koju string sadrži. Ukoliko string pročita iz serijskog monitora poruku **“LED ON”**, ugrađena led dioda kod Arduina svetli i u serijskom monitoru se ispisuje **“ACK: LED ON”**. Ovo simulira slanje ACK bita nazad koji naglasava da je LED ukljucen. Nakon toga slede jos dva uslova koji proveravaju da li je string cmd procitao iz serijskog monitora **“LED OFF”** ili **“STATUS?”** gde ovaj prvi signal isključuje diodu i ispisuje nazad poruku u serijskom monitoru da je led iskljucen i kod statusa javlja u da li digitalni pin 13 ima stanje logicke 1 ili 0. Ukoliko je logicka jedinica onda je LED ukljucen, u suprotnom nije. Ukoliko nijedna od komandi nije ucitana, dolazi do poruke koja naglašava da je uneta nepoznata komanda.

Opis Raspberry pi (python) programa

Cilj Raspberry Pi programa je da prikazuje ono što Arduino mikrokontroler šalje, da omogućava korisniku da preko **shell-a** unosi komande koje se šalju nazad Arduinu kao string. Na raspberry Pi operativnom sistemu, postoji fabrički instalirano razvojno okruženje koje je namenjeno često za rad sa python programskim jezikom. To je **Thonny IDE.**



Na početku Python koda vrši se **importovanje**, odnosno dodavanje biblioteka (modula) koje su potrebne za rad sa serijskim monitorom, nitima i Raspberry Pi mikroračunarom. To se radi uz pomoć ključne reči `import`.

- **`serial` modul** koristimo za rad sa **serijskom komunikacijom**, odnosno za čitanje, dekodiranje, enkodovanje i slanje podataka.
- **`threading` modul** koristimo za rad sa **nitima**, kada želimo da se više radnji izvršava istovremeno.
- **`Rpi.GPIO` modul** služi za rad sa **GPIO pinovima Raspberry Pi mikroračunara**.

Ovde se koristi ključna reč `as`, pa se modul importuje pod drugim imenom. U primeru se koristi ime `GPIO`, pa se umesto `Rpi.GPIO` u kodu piše samo `GPIO`.

Nakon dodavanje modula, sledi pisanje funkcije koja služi za prijem podataka odnosno poruka. Definiše se uz pomoć ključne reči def zatim naziv funkcije, zagrade i dve tačke na kraju. U ovom slučaju naša funkcija se naziva receive. Nakon toga važno je paziti na identaciju odnosno uvlačenje dela koda koji se koristi u funkciji. Postiže se klikom na TAB taster ili četiri razmaka. Kod koji je u ravni sa ključnom reči def se ne nalazi u funkciji i tako znamo koji deo koda je u funkciji a koji nije. Unutar funkcije imamo beskonačnu petlju (while True) u kojoj koristimo try i except blokove za obradu izuzetaka. U try bloku definisemo promenljivu data koja čita liniju po liniju sa serijskog monitora, vrši dekodiranje i uklanjanje prazne karaktere sa početka i kraja stringa. Zatim se proverava kroz uslov da li je data True i ukoliko jeste ispisujemo u **shellu [Arduino], data.** Trenutno u ovom kodu ukoliko se desi izuzetak neće da se obradi ali zato neće pući program s obzirom da je beskonačna petlja. Pass ključna reč se postavlja u delu koda u python programskom jeziku onuda gde želimo da definišemo deo koda koji trenutno ne radi ništa i prazan je ali ne smemo ga ostaviti tako jer će javiti grešku. Nakon toga sledi pokretanje prijemnog nita tako sto koristimo Thread klasu gde imamo target parametar koji poziva funkciju receive i vrši pokretanje nita uz pomoć funkcije **start**. U sledećem delu koda imamo obradu izuzetka koji treba da izvrši prekid programa tako sto se zadrži CTRL + C taster. Unutar glavne petlje definišemo promenljivu cmd koja omogućava korisnički unos iz **shella,** i nakon toga ispisuje unos i vrši enkodovanje. Na kraju ukoliko se desi izuzetak (CTRL + C) onda se poziva funkcija cleanup iz RPi.GPIO modula jkoja služi da vrati sve GPIO pinove u početno stanje nakon što program završi rad. Ukoliko se ova funkcija ne pozove, pinovi ostaju u stanju koje smo poslednje postavili (npr OUTPUT HIGH), i sledeći put kada se program pozove može se dobiti WARNING.

Primer kako finalna vežba radi:

Kada se Python program pokrene, trebalo bi da se vidi:

[Arduino]: ARDUINO_OK

[Arduino]: ARDUINO_OK

...

Korisnik upiše:

LED ON

Arduino odgovori:

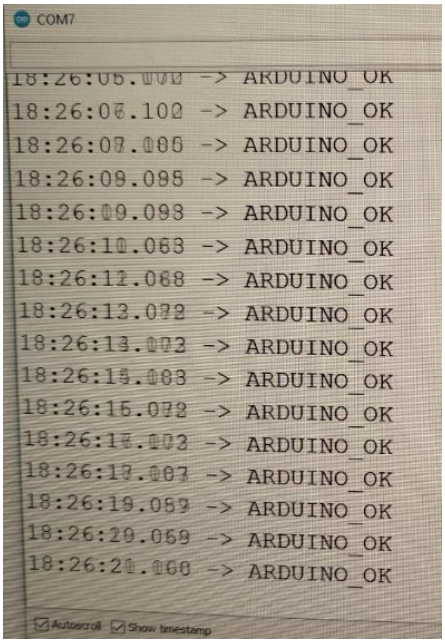
[Arduino]: ACK: LED ON

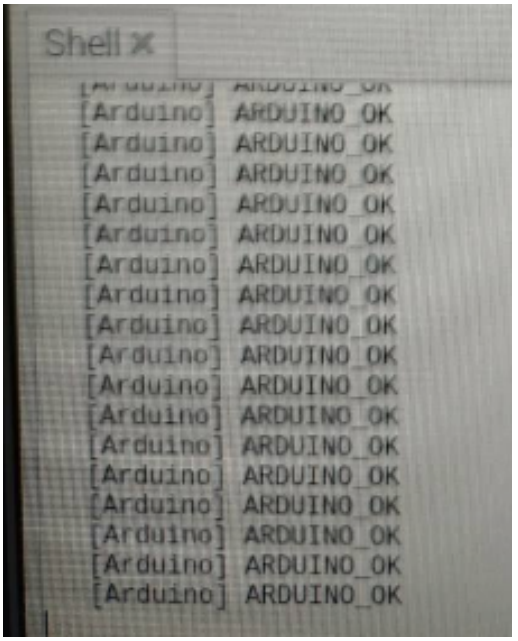
Zatim:

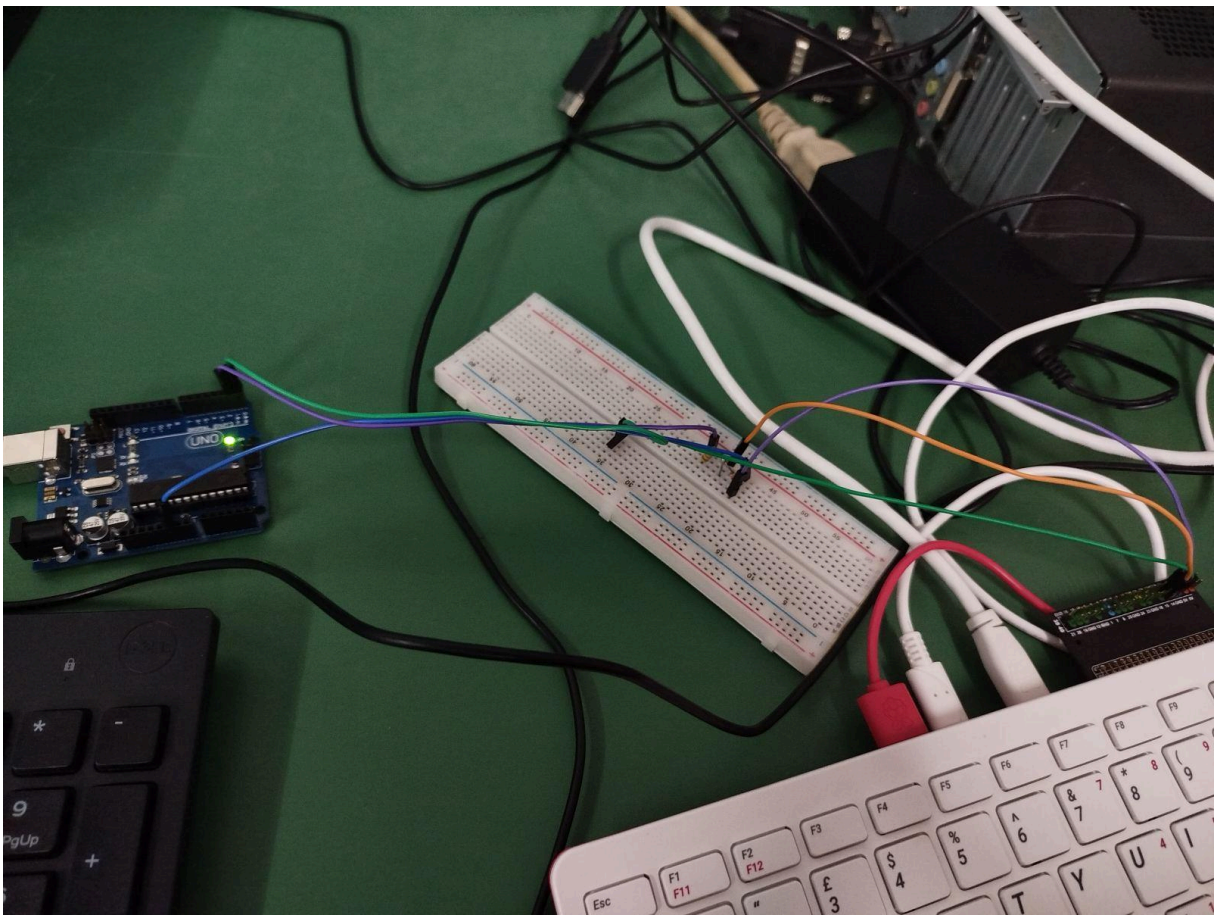
STATUS?

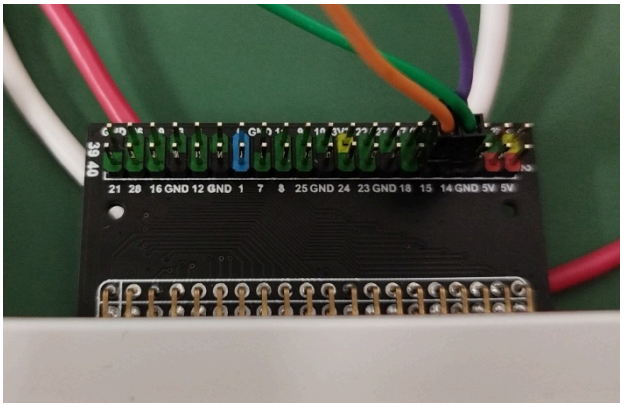
Odgovor:

[Arduino]: STATUS: LED=1











Na zadnjoj slici je prikazan GPIO header koji služi kao olakšica za spajanje GPIO pinova sa raspberry Pi mikroračunara. Pogotovo može biti od pomoći s obzirom da je u pitanju Rpi koji se nalazi unutar kućišta koje predstavlja tastatura, pa je teže prići do primarnog GPIO hedera. Pored te olakšice, takodje primarni GPIO header kod Rpi-a nema oznake pa može biti malo frustrirajuće za povezivanje.